## EXPERIMENT 4

## Nivass C R - 241801193

Implement a Convolutional Neural Network (CNN) from scratch to classify images. Train the network using a dataset of labeled images and evaluate its performance. Visualize the learned filters in the convolution layers.

```
import tensorflow as tf
import matplotlib.pyplot as plt

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

x_train = x_train / 255.0
x_test = x_test / 255.0

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(32, 32, 3)
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),
    tf.keras.layers.MaxPooling2D((2, 2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy")
plt.legend()
plt.show()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 1480s 9us/step
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 30, 30, 32) | 896 |
| max_pooling2d (MaxPooling2D) | (None, 15, 15, 32) | 0 |
| conv2d_1 (Conv2D) | (None, 13, 13, 64) | 18,496 |
| max_pooling2d_1 (MaxPooling2D) | (None, 6, 6, 64) | 0 |
| flatten (Flatten) | (None, 2304) | 0 |
| dense (Dense) | (None, 128) | 295,040 |
| dense_1 (Dense) | (None, 10) | 1,290 |

```
 Total params: 315,722 (1.20 MB)
 Trainable params: 315,722 (1.20 MB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 51s 71ms/step - accuracy: 0.4668 - loss: 1.4897 - val_accuracy: 0.5516 - val_loss: 1.2677
Epoch 2/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 47s 67ms/step - accuracy: 0.6033 - loss: 1.1251 - val_accuracy: 0.5888 - val_loss: 1.1365
Epoch 3/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 46s 66ms/step - accuracy: 0.6544 - loss: 0.9905 - val_accuracy: 0.6616 - val_loss: 0.9905
Epoch 4/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 48s 68ms/step - accuracy: 0.6883 - loss: 0.8972 - val_accuracy: 0.6690 - val_loss: 0.9444
Epoch 5/5
704/704 ━━━━━━━━━━━━━━━━━━━━ 48s 68ms/step - accuracy: 0.7133 - loss: 0.8278 - val_accuracy: 0.6808 - val_loss: 0.9199
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - accuracy: 0.6803 - loss: 0.9440
Test Loss: 0.9439809322357178
Test Accuracy: 0.6802999973297119
```

